

■ 강의명: CSCI E-89: Introduction to AI

■ 주차: Lecture 2

■ 교수명: Prof. John Doe

■ 목적: 인공지능 핵심 개념, 신경망, RNN, TF-IDF 이해 및 실습

Contents

□ 핵심 요약

본 문서는 CSCI89 Lecture 2 강의 자료를 통합하여 인공지능의 핵심 개념을 설명합니다. 신경망의 순전파와 역전파 원리, 학습률의 중요성, 순환 신경망(RNN)의 구조와 한계, LSTM 및 GRU와 같은 개선 모델, 그리고 텍스트 분류를 위한 TF-IDF 기법과 T-검정 기반 특징 선택 방법을 다룹니다. 실제 특히 데이터 분류 및 시계열 예측 예시를 통해 이론을 실습에 적용하는 방법을 제시하며, 초심자도 완벽히 이해할 수 있도록 상세한 설명과 예시를 제공합니다.

1 개요

이 문서는 CSCI E-89: Introduction to Artificial Intelligence 과목의 두 번째 강의 내용을 담고 있습니다. 신경망의 기본적인 동작 원리부터 시작하여, 복잡한 시퀀스 데이터를 처리하는 순환 신경망(RNN)과 그 변형들, 그리고 텍스트 데이터를 수치화하는 TF-IDF 기법까지 폭넓게 다룹니다. 각 개념은 비전공자도 쉽게 이해할 수 있도록 배경 설명, 직관적인 비유, 그리고 구체적인 예시를 포함하여 설명합니다.

2 용어 정리

인공지능 및 머신러닝 분야에서 사용되는 주요 용어들을 정리했습니다. 각 용어의 쉬운 설명과 함께 원어 및 추가적인 비고를 제공하여 이해를 돕습니다.

Table 1: 주요 용어 정리

용어	쉬운 설명	원어	비고
순전파	입력 데이터가 신경망을 통과하여 최종 출력을 계산하는 과정	Forward Propagation	신경망의 '계산' 단계
역전파	출력 오차를 기반으로 신경망의 가중치를 업데이트하기 위한 기울기를 계산하는 과정	Backward Propagation	신경망의 '학습' 단계
ReLU	입력이 양수이면 그대로 출력하고, 음수이면 0을 출력하는 활성화 함수	Rectified Linear Unit	가장 널리 사용되는 활성화 함수 중 하나
Softmax	다중 클래스 분류 문제에서 각 클래스에 속할 확률을 출력하는 활성화 함수	Softmax	모든 출력 확률의 합은 1
MSE	예측 값과 실제 값의 차이(오차)를 제곱하여 평균 낸 값	Mean Squared Error	회귀 문제에서 주로 사용되는 손실 함수
경사 하강법	손실 함수를 최소화하기 위해 함수의 기울기를 따라 파라미터를 점진적으로 업데이트하는 최적화 알고리즘	Gradient Descent	신경망 학습의 기본 원리
학습률 (α)	경사 하강법에서 파라미터 업데이트 시 기울기에 곱해지는 스케일 값	Learning Rate	하이퍼파라미터, 학습 속도 조절
하이퍼파라미터	모델 학습 전에 사용자가 직접 설정하는 값 (예: 학습률, 은닉층 수)	Hyperparameter	모델의 성능에 큰 영향
학습 가능한 파라미터	모델 학습 과정에서 데이터로부터 자동으로 최적화되는 값 (예: 가중치, 편향)	Trainable Parameter	모델의 '지식'을 구성
기울기 소실	역전파 과정에서 기울기 값이 너무 작아져 초기 층의 가중치가 거의 업데이트되지 않는 문제	Vanishing Gradient	RNN의 주요 문제점
기울기 폭주	역전파 과정에서 기울기 값이 너무 커져 가중치가 비정상적으로 업데이트되는 문제	Exploding Gradient	RNN의 또 다른 문제점
RNN	시퀀스 데이터를 처리하기 위해 이전 단계의 정보를 현재 단계로 전달하는 신경망	Recurrent Neural Network	시계열, 자연어 처리
LSTM	RNN의 기울기 소실 문제를 해결하기 위해 고안된 복잡한 구조의 순환 신경망	Long Short-Term Memory	장기 의존성 학습에 효과적
GRU	LSTM보다 간소화된 구조를 가지면서도 유사한 성능을 내는 순환 신경망	Gated Recurrent Unit	파라미터 수가 적어 효율적
TF-IDF	문서 내 단어의 중요도를 측정하는 통계적 수치	Term Frequency-Inverse Document Frequency	텍스트 벡터화 기법
T-검정	두 그룹 간의 평균 차이가 통계적으로 유의미한지 검정하는 통계 방법	T-test	특정 선택에 활용 가능
Epoch	전체 훈련 데이터셋을 신경망에 한 번 통과시키는 과정	Epoch	여러 번의 가중치 업데이트 포함
미니 배치	전체 훈련 데이터셋을 작은 묶음으로 나누어 한 번에 처리하는 데이터 단위	Mini-batch	효율적인 학습을 위한
Dropout	신경망 학습 시 과적합을 방지하기 위해 무작위로 뉴런의 연결을 끊는 정규화 기법	Dropout	훈련 시에만 적용
RMSprop	학습률을 파라미터별로 동적으로 조절하여 수렴을 돕는 최적화 알고리즘	Root Mean Square Propagation	Adam과 함께 널리 사용
Adam	RMSprop과 Momentum을 결합하여 학습률을 효율적으로 조절하는 최적화 알고리즘	Adaptive Moment Estimation	가장 널리 사용되는 최적화 알고리즘

3 핵심 개념 및 원리

3.1 신경망의 기본 동작: 순전파 (Forward Propagation)

신경망은 입력 데이터를 받아 여러 층을 거쳐 최종 출력을 만들어냅니다. 이 과정을 **순전파(Forward Propagation)**라고 합니다. 마치 공장에서 원재료(입력)가 여러 공정(신경망 층)을 거쳐 완제품(출력)이 되는 것과 같습니다.

1. **한 줄 핵심 요약:** 입력 데이터가 신경망의 각 층을 순서대로 통과하며 계산되어 최종 출력을 생성하는 과정입니다.
2. **직관적 예시/비유:** 계산기를 사용하여 복잡한 수식을 단계별로 푸는 것과 같습니다. 각 단계의 결과가 다음 단계의 입력이 됩니다.
3. **기술적 정확한 설명:** 신경망의 각 뉴런은 이전 층의 출력에 가중치(Weight, W)를 곱하고 편향(Bias, b)을 더한 후, 활성화 함수(Activation Function, f)를 적용하여 자신의 출력을 계산합니다. 이 출력이 다음 층의 입력이 됩니다. 최종 층의 출력이 신경망의 최종 예측 값($\hat{y}$)이 됩니다.

$$\text{뉴런 출력} = f\left(\sum(\text{이전 층 출력} \times W) + b\right)$$

□ 예제

ReLU (Rectified Linear Unit) 활성화 함수 ReLU는 신경망에서 가장 흔히 사용되는 활성화 함수 중 하나입니다.

- **정의:** 입력 값이 0보다 크면 그 값을 그대로 출력하고, 0보다 작거나 같으면 0을 출력합니다.
- **수식:** $f(z) = \max(0, z)$
- **특징:**
 - **양수 입력:** 입력이 양수일 때는 기울기가 1이므로 정보 손실 없이 전달됩니다.
 - **음수 입력:** 입력이 음수일 때는 기울기가 0이므로 해당 뉴런은 비활성화됩니다. 이는 신경망을 희소하게 만들어 계산 효율을 높일 수 있습니다.

□ 예제

퀴즈 1번: 순전파 계산 예시 주어진 신경망에서 최종 출력 $\hat{y}$ 를 계산하는 과정입니다. 활성화 함수는 ReLU를 사용합니다.

1단계: 첫 번째 은닉 뉴런 U_1 계산 U_1 은 입력 X_1, X_2 와 편향 $W_{01}^{(1)}$ 을 받습니다.

- $W_{01}^{(1)} = -1.2$ (편향)
- $W_{11}^{(1)} = 0.1$
- $W_{21}^{(1)} = 0.5$
- $X_1 = 1.3$
- $X_2 = 0.7$

선형 결합 $Z_1 = W_{01}^{(1)} + W_{11}^{(1)} \cdot X_1 + W_{21}^{(1)} \cdot X_2$
 $Z_1 = -1.2 + (0.1 \cdot 1.3) + (0.5 \cdot 0.7) = -1.2 + 0.13 + 0.35 = -0.72$
 활성화 함수 적용 $U_1 = f(Z_1) = \max(0, -0.72) = 0$ **결과:** $U_1 = 0$

2단계: 두 번째 은닉 뉴런 U_2 계산 U_2 는 입력 X_1, X_2 와 편향 $W_{02}^{(1)}$ 을 받습니다.

- $W_{02}^{(1)} = 0.9$ (편향)
- $W_{12}^{(1)} = 0.8$

- $W_{22}^{(1)} = 0.3$

- $X_1 = 1.3$

- $X_2 = 0.7$

선형 결합 $Z_2 = W_{02}^{(1)} + W_{12}^{(1)} \cdot X_1 + W_{22}^{(1)} \cdot X_2$ $Z_2 = 0.9 + (0.8 \cdot 1.3) + (0.3 \cdot 0.7) = 0.9 + 1.04 + 0.21 = 2.15$

활성화 함수 적용 $U_2 = f(Z_2) = \max(0, 2.15) = 2.15$ **결과:** $U_2 = 2.15$

3단계: 최종 출력 $\hat{y}$ 계산 $\hat{y}$ 는 은닉 뉴런 U_1, U_2 와 편향 $W_0^{(2)}$ 을 받습니다.

- $W_0^{(2)} = 0.2$ (편향)

- $W_1^{(2)} = 0.8$

- $W_2^{(2)} = 1.2$

- $U_1 = 0$

- $U_2 = 2.15$

선형 결합 $Z_{\hat{y}} = W_0^{(2)} + W_1^{(2)} \cdot U_1 + W_2^{(2)} \cdot U_2$ $Z_{\hat{y}} = 0.2 + (0.8 \cdot 0) + (1.2 \cdot 2.15) = 0.2 + 0 + 2.58 = 2.78$

활성화 함수 적용 $\hat{y} = f(Z_{\hat{y}}) = \max(0, 2.78) = 2.78$ **최종 결과:** $\hat{y} = 2.78$

가중치(W)와 편향(Bias)의 역할

- **가중치 (W):** 입력 신호의 중요도를 조절합니다. 가중치가 크면 해당 입력이 출력에 미치는 영향이 커집니다.
- **편향 (Bias):** 뉴런의 활성화 임계값을 조절합니다. 입력 신호가 없어도 뉴런이 활성화될 수 있도록 하거나, 특정 임계값 이상이어야 활성화되도록 돕습니다. 이는 모델이 데이터를 더 유연하게 학습할 수 있도록 합니다.
- **학습 가능한 파라미터 (Trainable Parameters):** 가중치와 편향은 신경망이 데이터를 통해 스스로 학습하여 최적의 값을 찾아내는 '학습 가능한 파라미터'입니다. 사용자가 직접 설정하는 '하이퍼파라미터'와는 다릅니다.

3.2 신경망 학습의 핵심: 역전파 (Backward Propagation)

신경망이 예측한 값과 실제 값 사이에 오차가 발생하면, 이 오차를 줄이기 위해 신경망의 가중치와 편향을 조정해야 합니다. 이 조정을 위해 각 파라미터가 오차에 얼마나 기여했는지 계산하는 과정이 **역전파 (Backward Propagation)**입니다.

1. **한 줄 핵심 요약:** 신경망의 출력 오차를 기반으로, 연쇄 법칙을 사용하여 각 가중치와 편향에 대한 손실 함수의 기울기를 효율적으로 계산하는 알고리즘입니다.
2. **직관적 예시/비유:** 복잡한 기계에서 최종 결과물이 잘못 나왔을 때, 어느 부품(가중치)이 얼마나 잘못 되었는지 역으로 추적하여 고치는 과정과 같습니다.
3. **기술적 정확한 설명:** 역전파는 경사 하강법 (Gradient Descent)과 함께 사용되어 신경망을 학습시킵니다. 손실 함수 (L)의 값을 최소화하기 위해 각 가중치 (W)에 대한 손실 함수의 기울기 ($\frac{\partial L}{\partial W}$)를 계산합니다. 이때, 순전파 과정에서 계산된 중간 값들을 재사용하여 계산 효율을 극대화합니다.

□ 예제

연쇄 법칙 (Chain Rule)을 이용한 기울기 계산 최종 출력 $\hat{y}$ 에 대한 가중치 W 의 기울기를 계산할

때 연쇄 법칙을 사용합니다. 예를 들어, $\frac{\partial \hat{y}}{\partial W_1^{(2)}}$ 를 계산하려면:

$$\frac{\partial \hat{y}}{\partial W_1^{(2)}} = \frac{\partial \hat{y}}{\partial Z_{\hat{y}}} \cdot \frac{\partial Z_{\hat{y}}}{\partial W_1^{(2)}}$$

여기서 $\frac{\partial \hat{y}}{\partial Z_{\hat{y}}}$ 는 활성화 함수의 미분(f') 이고, $\frac{\partial Z_{\hat{y}}}{\partial W_1^{(2)}}$ 는 U_1 이 됩니다.

$$\frac{\partial \hat{y}}{\partial W_1^{(2)}} = f'(Z_{\hat{y}}) \cdot U_1$$

만약 $U_1 = 0$ 이라면, $\frac{\partial \hat{y}}{\partial W_1^{(2)}}$ 도 0이 되어 해당 가중치는 업데이트되지 않습니다. 이처럼 순전파의 중간 결과(U_1)를 역전파에 재사용하는 것이 핵심입니다.

손실 함수 (Loss Function)의 미분 신경망 학습의 궁극적인 목표는 손실 함수를 최소화하는 것입니다. 손실 함수는 예측 값($\hat{y}$)과 실제 값(y)의 차이를 측정합니다.

- **전체 기울기:** $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial W}$
- **MSE (Mean Squared Error) 예시:** 만약 손실 함수 $L = (\hat{y} - y)^2$ 이라면, $\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y)$ 가 됩니다. Keras 나 TensorFlow 같은 딥러닝 프레임워크는 사용자가 손실 함수를 지정하면, 이처럼 분석적으로 미분된 $\frac{\partial L}{\partial \hat{y}}$ 값을 자동으로 계산하여 역전파에 활용합니다.

3.3 신경망 아키텍처 설계: 파라미터 수 계산

신경망의 복잡도는 파라미터(가중치와 편향) 수로 가늠할 수 있습니다. 파라미터 수가 많을수록 모델의 표현력은 높아지지만, 학습에 필요한 데이터와 계산량도 증가합니다.

1. **한줄 핵심 요약:** 신경망의 파라미터 수는 각 층의 뉴런 간 연결 수와 각 뉴런의 편향수를 합한 값입니다.
2. **직관적 예시/비유:** 복잡한 건물을 짓는 데 필요한 벽돌(연결)과 기둥(편향)의 총 개수를 세는 것과 같습니다.
3. **기술적 정확한 설명:** 완전 연결 신경망(Fully Connected Network, Dense Layer)에서 각 뉴런은 이전 층의 모든 뉴런과 연결됩니다. 따라서 파라미터 수는 다음과 같이 계산됩니다.
 - **가중치 수:** (이전 층 뉴런 수) $\times$ (현재 층 뉴런 수)
 - **편향 수:** (현재 층 뉴런 수)
 - **총 파라미터 수:** (이전 층 뉴런 수 + 1 (편향)) $\times$ (현재 층 뉴런 수)

□ 예제

퀴즈 2번: 완전 연결 신경망 파라미터 수 계산

- 입력층: 700개 입력 ($X_1, \dots, X_{700}$)
- 첫 번째 은닉층: 8개 뉴런
- 두 번째 은닉층: 8개 뉴런
- 출력층: 1개 뉴런 ($\hat{y}$)

1단계: 입력층 $\rightarrow$ 첫 번째 은닉층

- 입력 수: 700개

- 편향: 1개 (입력층에 편향은 없지만, 다음 층의 각 뉴런이 편향을 가짐)
- 첫 번째 은닉층 뉴런 수: 8개
- 파라미터 수: $(700 \text{ (입력)} + 1 \text{ (편향)}) \times 8 \text{ (뉴런)} = 701 \times 8 = 5608$

2단계: 첫 번째 은닉층 → 두 번째 은닉층

- 이전 층 (첫 번째 은닉층) 뉴런 수: 8개
- 편향: 1개
- 두 번째 은닉층 뉴런 수: 8개
- 파라미터 수: $(8 \text{ (이전 층 뉴런)} + 1 \text{ (편향)}) \times 8 \text{ (뉴런)} = 9 \times 8 = 72$

3단계: 두 번째 은닉층 → 출력층

- 이전 층 (두 번째 은닉층) 뉴런 수: 8개
- 편향: 1개
- 출력층 뉴런 수: 1개
- 파라미터 수: $(8 \text{ (이전 층 뉴런)} + 1 \text{ (편향)}) \times 1 \text{ (뉴런)} = 9 \times 1 = 9$

총 파라미터 수: $5608 + 72 + 9 = 5689$

Keras model.summary() 활용 실제 모델을 구축한 후 model.summary() 명령을 사용하면 각 층의 파라미터 수를 확인할 수 있습니다. 수동으로 계산한 값과 summary()의 결과가 일치하는지 확인하는 것은 신경망 아키텍처를 정확히 이해하고 있는지 점검하는 좋은 방법입니다.

3.4 출력층 활성화 함수 및 손실 함수 선택

신경망의 마지막 층(출력층)에서 어떤 활성화 함수를 사용하고, 어떤 손실 함수를 최소화할지는 해결하려는 문제의 종류(분류 또는 회귀)에 따라 달라집니다.

1. **한 줄 핵심 요약:** 문제 유형(분류/회귀)에 따라 출력층의 활성화 함수와 손실 함수를 적절히 선택해야 합니다.
2. **직관적 예시/비유:** 요리할 때 어떤 요리(문제 유형)를 만들지에 따라 마지막 양념(활성화 함수)과 맛 평가 기준(손실 함수)이 달라지는 것과 같습니다.
3. **기술적 정확한 설명:**
 - **분류 문제 (Classification):**
 - **다중 클래스 분류 (Multi-class Classification):** 16개의 클래스를 분류하는 문제처럼 여러 개의 범주 중 하나를 예측할 때 사용합니다.
 - * **출력층 뉴런 수:** 분류할 클래스 수와 동일하게 설정합니다 (예: 16개 클래스 → 16개 뉴런).
 - * **활성화 함수:** Softmax를 사용합니다. Softmax는 각 뉴런의 출력을 0과 1 사이의 확률 값으로 변환하며, 모든 출력 확률의 합이 1이 되도록 합니다.
 - * **손실 함수:** Categorical Cross-Entropy를 주로 사용합니다.
 - **이진 클래스 분류 (Binary Classification):** 두 개의 범주 중 하나를 예측할 때 사용합니다.
 - * **출력층 뉴런 수:** 1개 또는 2개.
 - * **활성화 함수:** 1개 뉴런일 경우 Sigmoid를 사용합니다 (0과 1 사이의 확률 출력). 2개 뉴런일 경우 Softmax를 사용할 수도 있습니다.
 - * **손실 함수:** Binary Cross-Entropy를 주로 사용합니다.

- **회귀 문제 (Regression):** 연속적인 숫자 값을 예측할 때 사용합니다 (예: 주택 가격 예측).
 - 출력층 뉴런 수: 예측하려는 값의 차원과 동일하게 설정합니다 (예: 하나의 숫자 예측 → 1 개 뉴런).
 - 활성화 함수: 일반적으로 활성화 함수를 사용하지 않거나 **선형 활성화 함수 (Linear Activation Function)**를 사용합니다.
 - 손실 함수: **Mean Squared Error (MSE)** 또는 **Mean Absolute Error (MAE)**를 주로 사용합니다.

3.5 신경망 학습의 조절자: 학습률 (Learning Rate, α)

학습률(α)은 경사 하강법(Gradient Descent)에서 손실 함수의 기울기를 따라 파라미터를 업데이트할 때, 한 번에 얼마나 크게 이동할지를 결정하는 하이퍼파라미터입니다.

1. **한 줄 핵심 요약:** 학습률은 신경망이 손실 함수의 최저점을 찾아가는 과정에서 한 걸음씩 움직이는 보폭을 결정하는 값입니다.
2. **직관적 예시/비유:** 산 정상에서 가장 낮은 계곡으로 내려갈 때, 한 걸음의 크기를 조절하는 것과 같습니다. 너무 작으면 시간이 오래 걸리고, 너무 크면 계곡을 건너뛰거나 엉뚱한 곳으로 갈 수 있습니다.
3. **기술적 정확한 설명:** 경사 하강법의 파라미터 업데이트 공식은 다음과 같습니다.

$$W_{\text{새로운}} = W_{\text{이전}} - \alpha \times \nabla J(W_{\text{이전}})$$

여기서 W 는 가중치 벡터, α 는 학습률, $\nabla J(W)$ 는 손실 함수 J 의 기울기 벡터입니다.

Figure 1: 학습률에 따른 손실 함수의 변화 (개념도)

학습률 α 값에 따른 학습 양상

- **α 가 너무 작을 때 (Small Learning Rate):**
 - 문제점: 파라미터 업데이트 보폭이 너무 작아 손실 함수의 최저점에 도달하는 데 매우 오랜 시간이 걸립니다. 심지어 학습이 거의 진행되지 않아 초기 값에 고착될 수도 있습니다.
 - 시각적 특징: 손실 함수 그래프가 거의 평평하게 유지되거나 매우 느리게 감소합니다.
 - 예상 오해: 모델이 학습되지 않는다고 오해할 수 있지만, 실제로는 너무 느리게 학습되고 있는 것입니다.
- **α 가 최적일 때 (Optimal Learning Rate):**
 - 특징: 적절한 보폭으로 손실 함수의 최저점을 향해 효율적으로 이동합니다. 수렴 속도가 빠르고 안정적으로 최적점에 도달합니다.
 - 시각적 특징: 손실 함수 그래프가 꾸준히 감소하다가 안정적으로 수렴합니다.
 - 실제: 완벽한 '최적'은 찾기 어렵지만, 충분히 좋은 값을 통해 효율적인 학습이 가능합니다.
- **α 가 너무 클 때 (Large Learning Rate):**
 - 문제점: 파라미터 업데이트 보폭이 너무 커서 손실 함수의 최저점을 건너뛰거나, 오히려 손실 값이 증가하며 발산(Divergence)할 수 있습니다.
 - 시각적 특징: 손실 함수 그래프가 불안정하게 진동하거나 급격히 증가합니다.
 - 예상 오해: 학습이 전혀 안 된다고 생각할 수 있지만, 실제로는 너무 과도하게 업데이트되어 최적점을 찾지 못하고 있는 것입니다.

- **진동하며 수렴:** 때로는 α 가 약간 커서 최저점 주변에서 진동(Oscillation)하며 수렴하는 경우도 있습니다. 이는 완전히 발산하는 것보다는 낫지만, 최적의 수렴은 아닙니다.

주의사항

데이터 스케일링의 중요성 학습률의 기본값(예: 0.01 또는 0.001)은 다양한 문제에 적용될 수 있도록 설정되어 있습니다. 하지만 입력 데이터의 스케일이 매우 크거나 작으면 이 기본값이 적절하지 않을 수 있습니다.

- **문제:** 예를 들어, 주택 가격(수백만 달러)을 입력으로 사용하는데, 학습률이 작은 경우 가중치 업데이트가 거의 이루어지지 않을 수 있습니다.
- **해결책:** 입력 데이터를 **스케일링(Scaling)**하여 0과 1 사이 또는 평균 0, 표준편차 1(Z-score 표준화)과 같은 일정한 범위로 맞춰주는 것이 중요합니다. 이렇게 하면 기본 학습률로도 안정적인 학습이 가능해집니다.

학습률 조절 기법

- **학습률 스케줄링 (Learning Rate Scheduling):** 학습이 진행됨에 따라 학습률을 점진적으로 감소시키는 방법입니다. 초반에는 빠르게 탐색하고, 후반에는 미세 조정을 통해 안정적인 수렴을 돕습니다.
- **최적화 기법 (Optimizers):** Adam, RMSprop과 같은 고급 최적화 알고리즘들은 학습률을 단순히 고정된 값으로 사용하는 대신, 각 파라미터에 대해 학습률을 동적으로 조절하거나, 기울기 벡터의 방향을 조절(회전)하여 더욱 효율적인 학습을 가능하게 합니다.
 - **RMSprop (Root Mean Square Propagation):** 각 파라미터의 기울기 제곱의 이동 평균을 사용하여 학습률을 조절합니다. 기울기가 큰 파라미터는 학습률을 줄이고, 기울기가 작은 파라미터는 학습률을 늘려줍니다.
 - **Adam (Adaptive Moment Estimation):** RMSprop과 Momentum(이전 기울기 정보를 활용하여 업데이트 방향을 부드럽게 하는 기법)을 결합한 최적화 기법으로, 가장 널리 사용됩니다.

4 순환 신경망 (Recurrent Neural Networks, RNN)의 이해

일반적인 신경망(완전 연결 신경망, 합성곱 신경망 등)은 각 입력이 독립적이라고 가정합니다. 하지만 시계열 데이터나 자연어와 같은 시퀀스 데이터는 이전 정보가 현재 정보에 영향을 미치는 '순서'와 '맥락'이 중요합니다. 순환 신경망(RNN)은 이러한 시퀀스 데이터를 처리하기 위해 고안된 신경망입니다.

4.1 RNN의 필요성: 시퀀스 데이터 처리

1. **한 줄 핵심 요약:** RNN은 순서가 중요한 시퀀스 데이터의 장기적인 의존성을 학습하기 위해 고안된 신경망입니다.
 2. **직관적 예시/비유:** 문장을 이해할 때, 단어 하나하나를 독립적으로 보는 것이 아니라 앞선 단어들의 맥락을 기억하며 다음 단어를 이해하는 것과 같습니다.
 3. **기술적 정확한 설명:**
 - **시계열 데이터 (Time Series Data):** 주식 가격, 기온 변화와 같이 시간 순서에 따라 관측되는 데이터입니다.
 - **자연어 (Natural Language):** 단어들이 문장이라는 순서를 이루어 의미를 전달하는 데이터입니다. 각 단어는 벡터로 표현될 수 있습니다.
 - **예측 문제:** 이전 관측 값들($X_1, X_2, \dots, X_T$)을 기반으로 다음 관측 값(X_{T+1})을 예측하는 것이 일반적인 시퀀스 예측 문제입니다.
- 기존 모델의 한계**
- **자기회귀 (AutoRegressive, AR) 모델:** 고전적인 시계열 모델로, 다음 관측 값이 이전 관측 값들의 선형 함수라고 가정합니다.

$$X_{T+1} = \beta_0 + \beta_1 X_T + \beta_2 X_{T-1} + \dots$$

- **한계:** 선형 관계만 모델링 가능하며, 시계열의 분산이 시간에 따라 변하지 않는 '정상성(Stationarity)'을 가정합니다. 금융 데이터처럼 변동성이 크게 변하는 경우(예: GARCH 모델이 필요한 경우)에는 적합하지 않습니다.
- **완전 연결 신경망 (Fully Connected Network):** 시퀀스 데이터를 완전 연결 신경망으로 처리하려면, 모든 이전 관측 값들을 한꺼번에 입력으로 넣어야 합니다.
 - **문제점:** 시퀀스 길이가 길어지면 입력 뉴런 수가 폭증하고, 파라미터 수도 기하급수적으로 늘어나 학습이 매우 어려워집니다. 또한, 시퀀스 길이가 가변적일 때 유연하게 대처하기 어렵습니다.

4.2 순환 뉴런 (Recurrent Neuron)의 구조

RNN의 핵심은 가중치 공유(Weight Sharing)와 은닉 상태(Hidden State)입니다. 이는 시퀀스의 각 시간 단계에서 동일한 계산 로직(동일한 가중치)을 사용하고, 이전 시간 단계의 정보를 현재 시간 단계로 전달하여 맥락을 유지합니다.

1. **한 줄 핵심 요약:** 순환 뉴런은 각 시간 단계에서 동일한 가중치를 공유하며, 이전 시간 단계의 은닉 상태를 현재 시간 단계의 입력으로 받아 시퀀스의 맥락을 학습합니다.
2. **직관적 예시/비유:** 기억력이 있는 사람과 같습니다. 새로운 정보를 들을 때(현재 입력), 이전에 들었던 정보(은닉 상태)를 바탕으로 새로운 정보를 해석하고, 그 결과를 다음 정보 해석에 활용합니다.
3. **기술적 정확한 설명:** 순환 뉴런은 현재 시간 단계의 입력 X_t 와 이전 시간 단계의 은닉 상태 Y_{t-1} 을

받아 현재 시간 단계의 은닉 상태 Y_t 를 계산합니다. 이 과정에서 사용되는 가중치(W_X, W_Y)와 편향(b)은 모든 시간 단계에서 동일하게 공유됩니다.

$$Y_t = f(W_X \cdot X_t + W_Y \cdot Y_{t-1} + b)$$

여기서 f 는 활성화 함수입니다.

Figure 2: 순환 뉴런의 전개된 구조 (Unfolded RNN)

순환 뉴런의 전개 (Unfolding) 위 그림처럼, 하나의 순환 뉴런은 시간 축을 따라 여러 개의 복사본이 연결된 형태로 이해할 수 있습니다.

- 초기 은닉 상태 (Y_0): 시퀀스의 시작점에서는 이전 은닉 상태가 없으므로, 보통 0으로 초기화됩니다.
- 가중치 공유: 그림에서 W_X, W_Y, b 는 모든 시간 단계에서 동일하게 사용됩니다. 이것이 파라미터 수를 크게 줄이는 핵심입니다.
- 입력 (X_t): 각 시간 단계 t 에서 새로운 입력 X_t 가 들어옵니다.
- 출력 (Y_t): 각 시간 단계 t 에서 은닉 상태 Y_t 가 출력됩니다. 이 Y_t 는 다음 시간 단계의 입력으로 다시 사용됩니다.

□ 예제

단일 순환 뉴런의 파라미터 수 계산

- 입력 X_t 의 차원: 1 (스칼라)
- 이전 은닉 상태 Y_{t-1} 의 차원: 1 (스칼라)
- 현재 은닉 상태 Y_t 의 차원: 1 (스칼라)
- X_t 에 대한 가중치 (W_X): 1개
- Y_{t-1} 에 대한 가중치 (W_Y): 1개
- 편향 (b): 1개
- 총 파라미터 수: $1 + 1 + 1 = 3$ 개

이처럼 단일 순환 뉴런은 시퀀스 길이에 관계없이 고정된 수의 파라미터를 가집니다.

4.3 RNN 학습의 난제: 기울기 소실 및 폭주 (Vanishing/Exploding Gradient)

RNN은 장기 의존성(Long-term Dependencies)을 학습하는 데 이론적으로 적합하지만, 실제 학습 과정에서는 기울기 소실(Vanishing Gradient) 또는 기울기 폭주(Exploding Gradient) 문제가 발생하여 어려움을 겪습니다.

1. **한 줄 핵심 요약:** 역전파 시 활성화 함수의 미분 값들이 반복적으로 곱해지면서 기울기가 너무 작아지거나(소실) 너무 커지는(폭주) 현상으로, RNN이 장기 의존성을 학습하기 어렵게 만듭니다.
2. **직관적 예시/비유:**
 - **기울기 소실:** 멀리 떨어진 과거의 사건(시퀀스 초반의 입력)이 현재 결과에 미치는 영향(기울기)이 너무 미미해져서, 마치 기억 상실증처럼 과거를 잊어버리는 것과 같습니다.
 - **기울기 폭주:** 아주 작은 변화가 기하급수적으로 증폭되어 통제 불능 상태가 되는 것과 같습니다.
3. **기술적 정확한 설명:** 역전파 과정에서 기울기는 연쇄 법칙에 따라 여러 활성화 함수의 미분 값과 가중치 값들이 곱해지면서 전달됩니다.
 - **기울기 소실 (Vanishing Gradient):**

- **원인:** Sigmoid나 Tanh와 같은 활성화 함수는 미분 값이 0과 1 사이(Sigmoid의 경우 최대 0.25)에 존재합니다. 이 작은 값들이 긴 시퀀스를 따라 반복적으로 곱해지면, 기울기 값이 0에 매우 가까워져 초기 시간 단계의 가중치 업데이트가 거의 이루어지지 않습니다.
- **결과:** RNN이 멀리 떨어진 과거의 정보를 학습하기 어려워져 장기 의존성을 포착하지 못합니다.
- **기울기 폭주 (Exploding Gradient):**
 - **원인:** 반대로, 활성화 함수의 미분 값이나 가중치 값이 1보다 크고, 이 값들이 반복적으로 곱해지면 기울기가 기하급수적으로 커져 발산할 수 있습니다.
 - **결과:** 가중치 업데이트가 너무 커져 모델이 불안정해지고 학습이 중단될 수 있습니다.
 - **해결책:** 기울기 클리핑(Gradient Clipping)을 통해 기울기 값이 특정 임계값을 넘지 않도록 제한하여 폭주를 방지할 수 있습니다.

4.4 순환 뉴런 층 (Layer of Recurrent Neurons)

단일 순환 뉴런만으로는 복잡한 시퀀스 패턴을 학습하기 어렵습니다. 여러 개의 순환 뉴런을 병렬로 배치하여 순환 뉴런 층(Layer of Recurrent Neurons)을 구성하면, 각 뉴런이 서로 다른 특징을 학습하며 더 풍부한 은닉 상태를 만들 수 있습니다.

1. **한 줄 핵심 요약:** 여러 개의 순환 뉴런을 병렬로 배치하여 시퀀스 데이터에서 더 복잡하고 다양한 특징을 학습하는 구조입니다.
2. **직관적 예시/비유:** 한 사람이 하나의 기억만 가지고 정보를 처리하는 것이 아니라, 여러 명의 전문가(각 뉴런)가 각자의 관점에서 정보를 처리하고 서로의 의견(교차 연결)을 교환하며 더 종합적인 결론(벡터 은닉 상태)을 내는 것과 같습니다.
3. **기술적 정확한 설명:** 순환 뉴런 층은 여러 개의 순환 뉴런으로 구성되며, 각 뉴런은 현재 입력 X_t 와 이전 시간 단계의 모든 은닉 상태(Y_{t-1} 벡터)를 입력으로 받습니다. 그리고 현재 시간 단계의 은닉 상태 벡터 Y_t 를 출력합니다.
 - **벡터 은닉 상태:** Y_t 는 더 이상 스칼라가 아닌 벡터가 됩니다. 각 요소는 층 내의 개별 순환 뉴런의 출력을 나타냅니다.
 - **교차 연결 (Cross-connections):** 층 내의 모든 순환 뉴런은 이전 시간 단계의 모든 순환 뉴런의 출력(은닉 상태 벡터의 모든 요소)을 입력으로 받습니다. 이는 완전 연결 신경망과 유사하게 층 내에서 정보가 교환됨을 의미합니다.
 - **입력 벡터 (X_t):** 입력 X_t 또한 스칼라가 아닌 벡터일 수 있습니다(예: 단어를 나타내는 임베딩 벡터). 이 경우, 각 X_t 의 요소는 층 내의 모든 순환 뉴런에 입력됩니다.

□ 예제

순환 뉴런 층의 파라미터 수 계산

- 입력 X_t 의 차원: 2 (벡터)
- 이전 은닉 상태 Y_{t-1} 의 차원: 2 (벡터, 층 내 뉴런 수)
- 현재 은닉 상태 Y_t 의 차원: 2 (벡터)

각 순환 뉴런은 다음과 같은 입력을 받습니다.

- X_t 의 2개 요소
- Y_{t-1} 의 2개 요소
- 편향 1개

따라서 각 뉴런당 $(2 + 2 + 1) = 5$ 개의 파라미터가 필요합니다. 층 내에 2 개의 뉴런이 있으므로, 총 파라미터 수는 $5 \times 2 = 10$ 개입니다.

Keras SimpleRNN 층 사용 Keras에서 SimpleRNN 층을 정의할 때 `units` 인수는 해당 층에 몇 개의 순환 뉴런이 있을지(즉, 은닉 상태 벡터의 차원)를 지정합니다.

```

1 from tensorflow.keras.layers import SimpleRNN, Dense
2 from tensorflow.keras.models import Sequential
3
4 # 입력데이터형태 : 샘플(수, 시간단계, 특징수)
5 # 예: (None, 200, 2) -> 200 시간단계, 각시간단계마다차원 2 특징벡터
6 input_shape = (200, 2)
7
8 model = Sequential([
9     SimpleRNN(units=3, activation='relu', input_shape=input_shape),
10    Dense(1) # 최종출력을위한완전연결층
11])
12 model.summary()

```

Listing 1: Keras SimpleRNN 층 정의 예시

위 예시에서 `SimpleRNN(units=3)`은 3개의 순환 뉴런으로 구성된 층을 의미하며, 은닉 상태 Y_t 는 3 차원 벡터가 됩니다.

5 RNN의 고급 변형: LSTM, GRU, 양방향 RNN

기울기 소실/폭주 문제와 장기 의존성 학습의 어려움을 극복하기 위해 다양한 RNN 변형 모델들이 개발되었습니다. 대표적으로 LSTM과 GRU는 '게이트(Gate)' 메커니즘을 도입하여 중요한 정보를 선택적으로 기억하고 불필요한 정보를 잊어버리도록 설계되었습니다.

5.1 장단기 기억 (Long Short-Term Memory, LSTM)

LSTM은 RNN의 기울기 소실 문제를 해결하고 장기 의존성을 효과적으로 학습하기 위해 1997년에 제안된 순환 신경망의 한 종류입니다.

1. **한 줄 핵심 요약:** 게이트 메커니즘을 통해 정보를 선택적으로 기억하고 잊어버리면서 장기적인 패턴을 학습할 수 있는 RNN의 고급 형태입니다.
2. **직관적 예시/비유:** 중요한 정보는 노트에 적어두고(장기 기억), 당장 필요한 정보는 잠시 머릿속에 두었다가(단기 기억) 활용하며, 불필요한 정보는 지워버리는 사람의 기억 시스템과 유사합니다.
3. **기술적 정확한 설명:** LSTM은 '셀 상태(Cell State, C_t)'라는 장기 기억 경로와 '은닉 상태(Hidden State, H_t)'라는 단기 기억 경로를 가집니다. 이 두 기억 경로는 세 가지 종류의 게이트(Gate)에 의해 조절됩니다.
 - **망각 게이트(Forget Gate):** 이전 셀 상태에서 어떤 정보를 잊어버릴지 결정합니다.
 - **입력 게이트(Input Gate):** 현재 입력에서 어떤 정보를 셀 상태에 저장할지 결정합니다.
 - **출력 게이트(Output Gate):** 셀 상태의 어떤 부분을 현재 은닉 상태로 출력할지 결정합니다.
 각 게이트는 시그모이드 활성화 함수를 사용하는 완전 연결 서브 네트워크로 구성되어 0과 1 사이의 값을 출력하며, 이를 통해 정보의 흐름을 제어합니다.

Figure 3: LSTM 셀의 내부 구조 (개념도)

LSTM의 파라미터 수 LSTM은 내부적으로 4개의 완전 연결 서브 네트워크(게이트)를 가집니다. 각 서브 네트워크는 입력 X_t , 이전 은닉 상태 H_{t-1} , 그리고 편향을 받습니다. 만약 LSTM(units=16)이라면, 각 서브 네트워크는 16개의 뉴런을 가지며, 은닉 상태와 셀 상태 모두 16차원이 됩니다. 이로 인해 SimpleRNN보다 훨씬 많은 파라미터를 가집니다.

활용 분야 및 장단점

- **장점:** 기울기 소실 문제를 효과적으로 완화하여 긴 시퀀스의 장기 의존성을 학습하는 데 매우 강력합니다.
- **단점:** 복잡한 내부 구조로 인해 SimpleRNN보다 계산 비용이 높고 파라미터 수가 많습니다.
- **활용:** 과거에는 기계 번역, 음성 인식 등 다양한 NLP 분야에서 지배적이었으나, 최근에는 Transformer 모델에 밀리는 경향이 있습니다. 하지만 짧은 시퀀스, 제한된 컴퓨팅 자원(예: 모바일 기기), 실시간 처리 등 특정 상황에서는 여전히 유용하게 사용됩니다.

5.2 게이트 순환 유닛 (Gated Recurrent Unit, GRU)

GRU는 LSTM의 복잡성을 줄이면서도 유사한 성능을 내기 위해 2014년에 제안된 순환 신경망의 또 다른 변형입니다.

1. **한 줄 핵심 요약:** LSTM보다 간소화된 게이트 구조를 가지며, 셀 상태와 은닉 상태를 하나로 통합하여 효율성을 높인 RNN 모델입니다.

2. 직관적 예시/비유: LSTM이 여러 개의 서류함(셀 상태)과 메모(은닉 상태)를 사용하는 반면, GRU는 하나의 서류함(통합된 은닉 상태)만으로 필요한 정보를 관리하는 것과 같습니다.
3. 기술적 정확한 설명: GRU는 LSTM의 셀 상태와 은닉 상태를 '은닉 상태(H_t)' 하나로 통합하고, 게이트 수도 두 개(업데이트 게이트, 리셋 게이트)로 줄였습니다.

- 업데이트 게이트 (Update Gate): 이전 은닉 상태를 얼마나 유지하고, 새로운 정보를 얼마나 받아 들일지 결정합니다.
- 리셋 게이트 (Reset Gate): 이전 은닉 상태 중 어떤 부분을 무시하고 새로운 정보를 계산할지 결정합니다.

이러한 간소화 덕분에 GRU는 LSTM보다 파라미터 수가 적고 계산 속도가 빠르면서도, 많은 경우 LSTM과 비슷한 성능을 보여줍니다.

GRU의 장점

- 효율성: LSTM보다 적은 파라미터와 빠른 계산 속도를 가집니다.
- 성능: 많은 벤치마크에서 LSTM과 유사하거나 때로는 더 나은 성능을 보여줍니다.

5.3 양방향 순환 신경망 (Bidirectional Recurrent Neural Networks)

일반적인 RNN은 시퀀스를 순방향으로만 처리하여 과거 정보만을 활용합니다. 하지만 어떤 시퀀스 데이터(특히 자연어)는 미래 정보(문장의 뒷부분)가 현재의 의미를 이해하는 데 중요할 수 있습니다. 양방향 순환 신경망(Bidirectional RNN)은 이러한 필요성을 충족시키기 위해 고안되었습니다.

1. 한 줄 핵심 요약: 시퀀스를 순방향과 역방향으로 동시에 처리하는 두 개의 RNN을 사용하여, 과거와 미래의 모든 맥락 정보를 활용하는 신경망입니다.
2. 직관적 예시/비유: 탐정이 사건을 조사할 때, 사건 발생 순서대로(순방향) 단서를 추적하는 동시에, 최종 결과(미래)를 바탕으로 역으로(역방향) 단서의 의미를 재해석하는 것과 같습니다.
3. 기술적 정확한 설명: 양방향 RNN은 다음과 같이 구성됩니다.
 - 순방향 RNN: 시퀀스를 $X_1, X_2, \dots, X_T$ 순서로 처리하여 은닉 상태 H_t^{forward} 를 생성합니다.
 - 역방향 RNN: 시퀀스를 $X_T, X_{T-1}, \dots, X_1$ 순서로 처리하여 은닉 상태 H_t^{backward} 를 생성합니다.
 - 출력 결합: 각 시간 단계 t 에서 순방향 은닉 상태 H_t^{forward} 와 역방향 은닉 상태 H_t^{backward} 를 결합(보통 연결(Concatenate))하여 최종 은닉 상태 $H_t^{\text{bidirectional}}$ 를 만듭니다.

이 최종 은닉 상태는 과거와 미래의 모든 정보를 담고 있으므로, 시퀀스의 특정 시점에서의 맥락을 더 풍부하게 이해할 수 있습니다.

□ 예제

언어 처리에서의 양방향 RNN 독일어와 같은 일부 언어에서는 문장의 부정(negation)이 문장 끝에 나타나는 경우가 있습니다. 이 경우, 문장의 시작 부분만 보고서는 전체 의미를 정확히 파악하기 어렵습니다. 양방향 RNN은 문장의 끝 부분 정보까지 활용하여 문장 전체의 의미를 더 정확하게 해석할 수 있습니다.

파라미터 수 증가 양방향 RNN은 두 개의 독립적인 RNN(순방향 및 역방향)을 사용하므로, 단방향 RNN에 비해 파라미터 수가 대략 두 배로 증가합니다. 이는 계산 비용 증가로 이어질 수 있습니다.

6 텍스트 데이터 전처리: TF-IDF (Term Frequency-Inverse Document Frequency)

텍스트 데이터를 신경망이나 머신러닝 모델에 입력하려면, 텍스트를 숫자로 변환하는 과정이 필요합니다. **TF-IDF (Term Frequency-Inverse Document Frequency)**는 문서 내 특정 단어의 중요도를 수치화하는 널리 사용되는 기법입니다.

6.1 TF-IDF의 개념 및 계산

1. **한 줄 핵심 요약:** TF-IDF는 특정 단어가 한 문서에 얼마나 자주 나타나고(TF), 전체 문서 집합에서 얼마나 희귀한지(IDF)를 종합하여 단어의 중요도를 측정하는 방법입니다.
2. **직관적 예시/비유:** 책에서 특정 단어가 얼마나 중요한지 평가하는 것과 같습니다. 한 챕터에 자주 나오면서(TF), 그 책 전체에서는 잘 나오지 않는(IDF) 단어일수록 그 챕터의 핵심 주제를 나타낼 가능성이 높습니다.
3. **기술적 정확한 설명:** TF-IDF는 두 가지 구성 요소의 곱으로 계산됩니다.
 - **용어 빈도 (Term Frequency, TF):** 특정 문서(d) 내에서 단어(t)가 나타나는 빈도를 측정합니다.

$$TF(t, d) = \frac{\text{문서 } d \text{ 내 단어 } t \text{의 출현 횟수}}{\text{문서 } d \text{ 내 전체 단어 수}}$$

- **역문서 빈도 (Inverse Document Frequency, IDF):** 단어가 전체 문서 집합에서 얼마나 희귀한지를 측정합니다. 희귀한 단어일수록 높은 IDF 값을 가집니다.

$$IDF(t, D) = \log \left(\frac{\text{전체 문서 수 } |D|}{\text{단어 } t \text{가 포함된 문서 수 } |\{d \in D : t \in d\}|} \right) + 1$$

(여기서 +1은 분모가 0이 되는 것을 방지하고, 로그 스케일을 조절하기 위한 일반적인 변형입니다.)

- **TF-IDF:** TF와 IDF의 곱입니다.

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

□ 예제

TF-IDF 계산 예시

- **문서 집합 (D):** 3개의 문서
 - 문서 1: "The cat sat on the mat." (총 6단어)
 - 문서 2: "The cat again did something." (총 5단어)
 - 문서 3: "Just some sentence." (총 3단어)

- **대상 단어:** "cat"

- **대상 문서:** 문서 1

1단계: TF 계산 (문서 1에서 "cat"의 용어 빈도)

- 문서 1에서 "cat" 출현 횟수: 1회
- 문서 1의 총 단어 수: 6개
- $TF(\text{"cat"}, \text{문서 1}) = 1/6$

2단계: IDF 계산 (전체 문서 집합에서 "cat"의 역문서 빈도)

- 전체 문서 수 ($|D|$): 3개
- "cat"이 포함된 문서 수: 문서 1, 문서 2 (총 2개)
- $IDF("cat", D) = \log(3/2) + 1 \approx 0.405 + 1 = 1.405$

3단계: TF-IDF 계산

- $TF-IDF("cat", \text{문서 1}, D) = (1/6) \times (\log(3/2) + 1) \approx 0.167 \times 1.405 \approx 0.234$

TF-IDF 벡터화 과정

- **훈련 (fit):** TfidfVectorizer는 훈련 데이터셋을 기반으로 모든 고유 단어(어휘 집)를 구축하고, 각 단어의 IDF 값을 계산합니다.
- **변환 (transform):** transform 메서드는 새로운 문서(훈련 또는 테스트)에 대해 TF를 계산하고, fit 과정에서 학습된 IDF 값을 적용하여 TF-IDF 벡터를 생성합니다. 테스트 데이터에 대해서는 IDF를 새로 계산하지 않고 훈련 데이터에서 학습된 IDF를 그대로 사용합니다.
- **정규화 (Normalization):** 일반적으로 TF-IDF 벡터는 L2 정규화(벡터의 길이를 1로 만들)를 거쳐 스케일이 조정됩니다.

주의사항

단어 순서 정보 손실 TF-IDF는 각 단어의 중요도를 측정하지만, 문서 내에서 단어들이 나타나는 순서 정보는 완전히 무시합니다. 이는 텍스트 분류와 같이 단어의 존재 여부가 중요한 문제에는 적합하지만, 문맥이나 문법이 중요한 문제(예: 기계 번역)에는 한계가 있습니다.

7 특허 데이터 분류 실습 (TF-IDF + 로지스틱 회귀/신경망)

실제 특허 데이터를 사용하여 TF-IDF와 머신러닝 모델을 이용해 문서 분류를 수행하는 과정을 살펴봅니다. 목표는 특정 특허가 자동차 산업과 관련이 있는지 여부를 분류하는 것입니다.

7.1 문제 정의 및 데이터셋 구성

1. **한 줄 핵심 요약:** 19세기 말에서 20세기 초의 특허 문서들을 자동차 산업 관련 여부에 따라 분류하는 이진 분류 문제입니다.
2. **데이터 소스:** Google Patents와 같은 공개 데이터베이스에서 수집된 특허 데이터 (출판 번호, 제목, 설명).
3. **훈련 데이터셋 생성:**
 - **긍정 레이블 (1):** 'Wheels within Wheels'와 같은 역사 서적에서 확인된 자동차 산업 관련 기업들이 출원한 특허들을 '자동차 산업 관련'으로 레이블 1을 부여합니다.
 - **부정 레이블 (0):** 전체 특허 중 무작위로 샘플링한 특허들에 대해 '자동차 산업 비관련'으로 레이블 0을 부여합니다. 이 방법은 100% 정확하지 않을 수 있지만, 방대한 특허 수에 비추어 대부분의 무작위 샘플은 자동차 산업과 무관할 것이라는 가정에 기반합니다.

7.2 차원 축소 및 특징 선택: T-검정 (T-test) 활용

특허 문서에는 수많은 고유 단어가 포함되어 있어, 이를 모두 특징으로 사용하면 모델의 차원이 너무 높아져 학습이 어렵고 과적합될 위험이 있습니다. **T-검정 (T-test)**을 사용하여 가장 중요한 단어들만 선별하여 차원을 축소합니다.

1. **한 줄 핵심 요약:** T-검정을 통해 자동차 산업 관련 특허와 비관련 특허 그룹 간에 TF-IDF 값이 통계적으로 유의미한 차이를 보이는 단어들을 선별하여 특징으로 사용합니다.
2. **직관적 예시/비유:** 두 그룹의 학생들에게 시험을 치르게 한 후, 어떤 문제(단어)가 두 그룹의 실력 (TF-IDF 값) 차이를 가장 잘 보여주는지 통계적으로 분석하는 것과 같습니다.
3. **기술적 정확한 설명:**
 - **T-검정의 목적:** 두 모집단(여기서는 '자동차 관련 특허'와 '비관련 특허' 그룹)의 평균이 통계적으로 유의미하게 다른지 여부를 검정합니다.
 - **절차:**
 - (a) **가설 설정:** 귀무가설(H_0): 두 그룹 간 특정 단어의 TF-IDF 평균은 같다. 대립가설(H_1): 두 그룹 간 특정 단어의 TF-IDF 평균은 다르다.
 - (b) **T-통계량 계산:** 두 그룹의 평균 차이를 표준 오차로 나눈 값입니다. 이 값이 클수록 평균 차이가 크다는 의미입니다.
 - (c) **P-값 (P-value) 계산:** T-통계량이 관측될 확률을 나타냅니다. P-값이 작을수록 귀무가설을 기각하고 대립가설을 채택할 강력한 증거가 됩니다. 즉, 두 그룹 간의 평균 차이가 우연히 발생했을 가능성이 낮다는 의미입니다.
 - **특징 선택:** 각 단어에 대해 T-검정을 수행하고 P-값을 계산합니다. P-값이 가장 작은 상위 N개의 단어(예: 제목에서 300개, 설명에서 600개)를 최종 특징으로 선택합니다. 이는 두 그룹을 가장 잘 구분하는 단어들이라고 볼 수 있습니다.

7.3 모델 학습 및 평가

TF-IDF 벡터화 및 T-검정을 통해 선별된 특징들을 사용하여 로지스틱 회귀 모델과 신경망 모델을 학습시키고 성능을 평가합니다.

1. **한 줄 핵심 요약:** TF-IDF 특징을 기반으로 로지스틱 회귀 및 신경망 모델을 훈련하고, 검증 정확도 (Validation Accuracy)를 통해 모델의 성능을 평가합니다.
2. **모델 유형:**
 - **로지스틱 회귀 (Logistic Regression):**
 - **제목 기반:** 제목에서 추출된 상위 300개 TF-IDF 특징 사용.
 - **설명 기반:** 설명에서 추출된 상위 600개 TF-IDF 특징 사용.
 - **결합 기반:** 제목(300개)과 설명(600개)의 TF-IDF 특징을 연결(Concatenate)하여 총 900개 특징 사용.
 - **신경망 (Neural Network):**
 - **아키텍처:** 3개의 은닉층 (각 16개 뉴런), 출력층 (2개 뉴런, Softmax 활성화).
 - **최적화:** RMSprop (또는 Adam) 옵티마이저.
 - **손실 함수:** Categorical Cross-Entropy.
 - **평가 지표:** 정확도 (Accuracy).
 - **정규화:** Dropout (과적합 방지).

훈련/검증 정확도 곡선 분석 모델 학습 시 훈련 정확도(Training Accuracy)와 검증 정확도(Validation Accuracy)를 함께 모니터링하는 것이 중요합니다.

- **훈련 정확도:** 훈련 데이터에 대한 모델의 성능. 에포크가 진행될수록 일반적으로 계속 증가합니다.
- **검증 정확도:** 모델이 보지 못한 새로운 데이터(검증 데이터)에 대한 성능. 훈련 정확도와 함께 증가하다가, 모델이 과적합(Overfitting)되기 시작하면 감소하거나 정체될 수 있습니다.
- **최적 모델 선택:** 검증 정확도가 가장 높은 시점의 모델을 선택하는 것이 가장 좋습니다. 훈련 정확도가 계속 높아지더라도 검증 정확도가 더 이상 개선되지 않거나 감소한다면, 과적합이 발생하고 있으므로 학습을 중단해야 합니다.

8 시계열 예측 실습 (RNN/LSTM)

RNN과 LSTM을 사용하여 GARCH 모델과 유사한 비선형 시계열 데이터를 예측하는 실습 과정을 다룹니다.

8.1 데이터 생성 및 준비

1. **한 줄 핵심 요약:** GARCH와 유사한 비선형 시계열 데이터를 생성하고, 이를 RNN 모델의 입력 형태로 변환하여 훈련 및 테스트 데이터셋을 구성합니다.
2. **시계열 생성 함수:** GARCH(Generalized AutoRegressive Conditional Heteroskedasticity) 모델은 금융 시계열의 변동성 클러스터링(Volatility Clustering)을 모델링하는 데 사용됩니다. 본 실습에서는 GARCH와 유사한 구조에 비선형 상호작용 항을 추가하여 더 복잡한 시계열을 생성합니다.

```

1 import numpy as np
2
3 def generate_garch_like_series(n_steps=1000, alpha=0.1, beta=0.8, gamma
    =0.05, interaction=0.1):
4     series = np.zeros(n_steps)
5     volatility = np.zeros(n_steps)
6     volatility[0] = 0.1 # 초기변동성
7
8     for t in range(1, n_steps):
9         # 와 GARCH 유사한변동성업데이트
10        volatility[t] = np.sqrt(alpha * series[t-1]**2 + beta * volatility[
            t-1]**2 + gamma)
11        # 비선형상호작용항추가
12        series[t] = np.random.normal(0, volatility[t]) + interaction *
            series[t-1] * volatility[t-1]
13    return series, volatility

```

Listing 2: GARCH 유사 시계열 생성 함수 (개념 코드)

3. **데이터셋 체크 분할:** RNN 모델은 고정된 길이의 시퀀스를 입력으로 받습니다. 따라서 생성된 시계열을 '입력 시퀀스'와 '예측할 다음 값'으로 구성된 청크(Chunk)로 분할해야 합니다.
 - 예시: `time_steps = 14` ,
4. **RNN 입력 형태 재구성:** Keras의 RNN 층은 입력 데이터를 (샘플 수, 시간 단계, 특징 수)의 3차원 형태로 기대합니다.
 - 문제: 1차원 시계열 데이터(예: (1000,))를 (샘플 수, 시간 단계) 형태로만 제공하면 오류가 발생합니다.
 - 해결: `reshape` 함수를 사용하여 (샘플 수, 시간 단계, 1) 형태로 명시적으로 특징 차원을 추가해야 합니다.
5. **훈련/테스트 데이터 분할:** 시계열 데이터는 시간 순서가 중요하므로, 무작위로 섞지 않고 시계열의 앞부분을 훈련 데이터로, 뒷부분을 테스트 데이터로 사용합니다 (예: 80% 훈련, 20% 테스트).

8.2 RNN/LSTM 모델 구축 및 학습

1. 한 줄 핵심 요약: SimpleRNN 또는 LSTM 층을 사용하여 시계열 예측 모델을 구축하고, Dropout 으로 과적합을 방지하며 학습시킵니다.
2. 모델 아키텍처 예시 (Keras):

```

1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import SimpleRNN, LSTM, Dense, Dropout
3
4 # 입력형태 : 시간( 단계, 특징수 )
5 # 예: (14, 1) -> 14 시간단계 , 각시간단계마다차원 1 특징
6 n_time_steps = 14
7 n_features = 1
8
9 model = Sequential([
10     # 첫번째 RNN 층: 전체시퀀스를다음층으로전달 (return_sequences=True)
11     SimpleRNN(units=2, activation='relu', input_shape=(n_time_steps,
12     n_features), return_sequences=True),
13     Dropout(0.1), # 과적합방지를위한 Dropout (10% 연결드롭 )
14     # 두번째 RNN 층: 마지막은닉상태만다음층으로전
15     달 (return_sequences=False)
16     SimpleRNN(units=4, activation='relu', return_sequences=False),
17     Dropout(0.1),
18     # 최종예측을위한 Dense 층회귀이므로 ( 활성화함수없음 )
19     Dense(1)
20 ])
21
22 model.compile(optimizer='adam', loss='mse') # 회귀문제이므로 MSE 손실함수사용
23 model.summary()

```

Listing 3: RNN 시계열 예측 모델 (개념 코드)

`return_sequences`

- 3. `return_sequences = True`: RNN . RNN .
- 3. `return_sequences = False`(): RNN . RNN (DenseLayer) .

Dropout (드롭아웃) 정규화

- 목적: 신경망의 과적합(Overfitting)을 방지하기 위한 정규화 기법입니다.
- 동작 방식: 훈련 과정에서 각 에포크마다 무작위로 일부 뉴런(또는 RNN의 경우 연결)을 비활성화(드롭)시킵니다. 이는 모델이 특정 뉴런에 과도하게 의존하는 것을 방지하고, 더 강건한 특징을 학습하도록 돕습니다.
- 훈련 시에만 적용: 드롭아웃은 훈련 시에만 적용되며, 테스트 또는 예측 시에는 모든 뉴런이 활성화됩니다.

학습 및 평가:

- 옵티마이저: Adam (효율적인 학습률 조절).
- 손실 함수: MSE (회귀 문제).
- 배치 크기 (Batch Size): 한 번의 가중치 업데이트에 사용되는 샘플 수.
- 검증 데이터 (Validation Data): 훈련 중 모델의 일반화 성능을 모니터링하기 위해 사용됩니다.

- **평가:** 훈련 및 테스트 데이터에 대한 MSE 값을 에포크별로 플로팅하여 모델의 학습 진행 상황과 과적합 여부를 확인합니다.

9 절차/방법

신경망 모델을 구축하고 학습시키는 일반적인 절차와 특정 문제 해결을 위한 단계별 방법을 정리했습니다.

9.1 신경망 모델 학습 및 평가 절차

1. 데이터 준비:

- 원본 데이터 수집 및 탐색.
- 결측치 처리, 이상치 제거 등 데이터 정제.
- 데이터 스케일링: 입력 특징들을 0-1 범위 또는 표준 정규 분포로 변환하여 학습 안정성 및 속도 향상.
- 데이터 분할: 훈련(Train), 검증(Validation), 테스트(Test) 세트로 분할. 시계열 데이터는 시간 순서 유지.

2. 모델 아키텍처 정의:

- 문제 유형(분류/회귀)에 따라 적절한 신경망 층(Dense, SimpleRNN, LSTM 등) 선택.
- 각 층의 뉴런 수(units), 활성화 함수(activation) 설정.
- 출력층의 뉴런 수와 활성화 함수를 문제에 맞게 설정 (예: 다중 분류 → 클래스 수, Softmax).
- `input_shape` .

3. 모델 컴파일:

- 옵티마이저(optimizer): Adam, RMSprop 등 선택.
- 손실 함수(loss): 문제 유형에 맞게 선택 (예: 회귀 → 'mse', 다중 분류 → 'categorical_crossentropy').
- 평가 지표(metrics): 훈련 중 모니터링할 지표 설정 (예: 'accuracy', 'mse').

4. 모델 학습 (model.fit()):

- 훈련 데이터(x_{train} ,
- 에포크(epochs): 전체 훈련 데이터를 몇 번 반복하여 학습할지 지정.
- 배치 크기(batch_size): .
- 검증 데이터(validation_data): .
- 콜백(callbacks): EarlyStopping, ModelCheckpoint 등을 사용하여 학습 과정 제어.

5. 모델 평가 (model.evaluate()):

- 테스트 데이터(x_{test} ,
- 훈련 및 검증 과정에서 사용되지 않은 완전히 새로운 데이터로 평가하여 모델의 실제 일반화 성능을 측정합니다.

6. 예측 (model.predict()):

- 학습된 모델을 사용하여 새로운 입력에 대한 예측 값을 생성합니다.

9.2 TF-IDF 기반 텍스트 벡터화 및 특징 선택 (T-test 활용)

1. 텍스트 데이터 로드 및 전처리:

- 텍스트 데이터(예: 특허 제목, 설명)를 불러옵니다.
- 불용어(stop words) 제거, 토큰화(tokenization), 표제어 추출(lemmatization) 등 기본적인 텍스트

전처리를 수행합니다.

2. **TF-IDF 벡터화 객체 생성 및 학습:**

- `TfidfVectorizer` 객체를 생성합니다.
- 훈련 데이터셋의 텍스트를 사용하여 `fit()` 메서드를 호출하여 어휘 집과 IDF 값을 학습시킵니다.

3. **TF-IDF 특징 추출:**

- `transform()` 메서드를 사용하여 훈련 데이터셋의 텍스트를 TF-IDF 벡터로 변환합니다.
- 각 단어(특징)에 대한 TF-IDF 값을 얻습니다.

4. **T-검정을 통한 특징 선택:**

- 훈련 데이터셋을 레이블(예: 0과 1)에 따라 두 그룹으로 나눕니다.
- 각 단어(TF-IDF 특징)에 대해 두 그룹 간의 TF-IDF 값 평균 차이에 대한 T-검정을 수행하고 P-값을 계산합니다.
- P-값이 가장 작은 상위 N개의 단어를 최종 특징으로 선택합니다.

5. **선택된 특징으로 데이터셋 재구성:**

- 선택된 N개의 단어에 해당하는 TF-IDF 값만 사용하여 훈련 및 테스트 데이터셋을 재구성합니다.
- 이를 통해 모델의 입력 차원을 줄이고 학습 효율을 높입니다.

10 실습/코드/오류와 해결

10.1 Keras 신경망 모델 정의 예시

다음은 Keras를 사용하여 간단한 완전 연결 신경망을 정의하는 코드 예시입니다.

```

1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import Dense
3
4 # 입력특징의수예    (: TF-IDF 특징개 900)
5 input_dim = 900
6
7 model = Sequential([
8     Dense(16, activation='relu', input_shape=(input_dim,)), # 첫번째은닉층
9     Dense(16, activation='relu'), # 두번째은닉층
10    Dense(16, activation='relu'), # 세번째은닉층
11    Dense(2, activation='softmax') # 출력층개 (2 클래스분류 , Softmax)
12 ])
13
14 model.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['
15     accuracy'])
16 model.summary()

```

Listing 4: Keras 완전 연결 신경망 모델 정의

10.2 TF-IDF Vectorizer 사용 예시

sklearn의 TfidfVectorizer를 사용하여 텍스트를 TF-IDF 벡터로 변환하는 예시입니다.

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2 import numpy as np
3
4 documents = [
5     "The cat sat on the mat.",
6     "The cat again did something.",
7     "Just some sentence but no cat."
8 ]
9
10 # TfidfVectorizer 객체생성
11 # min_df: 최소문서빈도너무    ( 희귀한단어제거 )
12 # max_df: 최대문서빈도너무    ( 흔한단어제거 )
13 vectorizer = TfidfVectorizer(min_df=1, max_df=1.0)
14
15 # 훈련데이터에 fit (IDF 값학습 )
16 vectorizer.fit(documents)
17
18 # 훈련데이터변환    (TF-IDF 벡터생성 )
19 tfidf_matrix = vectorizer.transform(documents)
20
21 print("어휘집:", vectorizer.get_feature_names_out())

```